

Towards A Universal Graph Structural Encoder

Jialin Chen

Department of Computer Science
Yale University
New Haven, CT, United States
jialin.chen@yale.edu

Haolan Zuo

Department of Computer Science
Yale University
New Haven, CT, United States
haolanzuo@gmail.com

Haoyu Wang

Department of Electrical and
Computer Engineering
Georgia Institute of Technology
Atlanta, GA, United States
haoyu.wang@gatech.edu

Siqi Miao

Department of Electrical and
Computer Engineering
Georgia Institute of Technology
Atlanta, GA, United States
siqi.miao@gatech.edu

Pan Li

Department of Electrical and
Computer Engineering
Georgia Institute of Technology
Atlanta, GA, United States
panli@gatech.edu

Rex Ying

Department of Computer Science
Yale University
New Haven, CT, United States
rex.ying@yale.edu

Abstract

Recent advancements in large-scale pre-training have shown the potential to learn generalizable representations for downstream tasks. In the graph domain, however, capturing and transferring structural information across different graph domains remains challenging, primarily due to the inherent differences in graph topological patterns across various contexts. For example, a social network's structure is fundamentally different from that of a product co-purchase graph. Additionally, most existing models struggle to capture the rich topological complexity of graph structures, leading to inadequate exploration of the graph embedding space. To address these challenges, we propose GFSE, a universal pre-trained graph encoder designed to capture transferable structural patterns across diverse domains such as the web graph, social networks, and citation networks. GFSE is the first cross-domain graph structural encoder pre-trained with multiple self-supervised learning objectives. Built on a Graph Transformer, GFSE incorporates attention mechanisms informed by graph structural information, enabling it to encode intricate multi-level and fine-grained topological features within complex graph structures. The pre-trained GFSE produces generic and theoretically expressive positional and structural encoding for graphs, which can be seamlessly integrated with various downstream graph feature encoders, including graph neural networks for vectorized features and Large Language Models (LLMs) for text-attributed graphs. Comprehensive experiments on synthetic and real-world datasets demonstrate GFSE's capability to significantly enhance the model's performance while requiring substantially less task-specific fine-tuning. Notably, GFSE achieves state-of-the-art performance in diverse evaluated cases spanning various graph models and datasets, highlighting its potential as a powerful and versatile encoder for graph-structured data.

CCS Concepts

• Computing methodologies → Supervised learning.

Keywords

Graph Representation Learning, Self-supervised Learning

ACM Reference Format:

Jialin Chen, Haolan Zuo, Haoyu Wang, Siqi Miao, Pan Li, and Rex Ying. 2026. Towards A Universal Graph Structural Encoder. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792656>

1 Introduction

The success of large-scale pre-trained models in domains such as natural language processing [1, 13, 79], computer vision [71, 72], audio [7, 93] has demonstrated the power of learning transferable representations across diverse datasets. However, developing such foundational models for the graph domain remains a significant challenge. Due to the inherent heterogeneity of graph-structured data across the domains, most prior graph pre-training efforts are highly specialized, focusing on narrow domains such as link structures in web graphs, user interactions in social networks, or relational facts in knowledge graphs [31, 88, 99]. These specialized models often rely heavily on domain-specific features, which limits their transferability and requires deep domain expertise to adapt.

This feature-centric approach overlooks a critical, unifying aspect of graphs: they exhibit rich structural patterns that are often shared across domains. For example, social networks and citation networks commonly exhibit small-world properties and dense community structures, while e-commerce graphs reveal hierarchical taxonomies. Despite the prevalence of these fundamental patterns, they are often secondary to domain-specific features, hindering the generalization capabilities of existing pre-trained models.

As tokens in NLP derive meaning from their context, nodes in graphs gain significance through their structural role. Therefore, we propose a paradigm shift that prioritizes learning universal structural attributes—domain-agnostic features that are shared across diverse graph domains. These attributes, such as motifs, node degrees, and hierarchical topologies, serve as the building blocks



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792656>

for transferable graph representations. The central challenge lies in developing a foundational model that can effectively capture these universal structural patterns while remaining adaptable for downstream tasks. Such a model would reduce the reliance on domain-specific engineering and unlock the potential for truly general-purpose graph representation learning.

Proposed work. To address the challenges of cross-domain pre-training and effectively capturing universal structural encoding, we propose **GFSE**, a **Graph Foundational Structural Encoder**, as shown in Figure 1. GFSE is pre-trained across diverse graph domains using multiple self-supervised pre-training tasks, including shortest path distance regression, motif counting, local community detection, and graph-level contrastive learning. Each pre-training task targets a critical and necessary aspect of graph structures, enabling GFSE to capture a comprehensive understanding of graph topology. GFSE employs a Graph Transformer enhanced with biased attention mechanisms. Notably, the relative positional encoding, derived from the random walk matrix, is explicitly integrated into the attention bias term. This design allows GFSE to effectively capture intricate structural dependencies among node pairs during pre-training, ensuring both efficiency and theoretically guaranteed expressiveness. GFSE’s versatility extends to various graph learning scenarios. The pre-trained GFSE can produce generic and expressive Positional and Structural Encodings (PSE) for topological tasks. In feature-enriched contexts, the generated PSE can seamlessly augment vectorized features or integrate with language models for text-attributed graphs, enabling GFSE to serve as a powerful component in graph foundation models.

The **contributions** of this work are threefold. (1) We propose GFSE, a unified cross-domain graph structural encoder pre-trained with four essential self-supervised learning objectives. Extensive experiments demonstrate the effectiveness of these pre-training tasks, resulting in state-of-the-art performance across diverse evaluated cases, encompassing various graph models and datasets. (2) We provide theoretical justification and empirical results demonstrating GFSE’s ability to generate expressive PSE. (3) GFSE serves as a plug-and-play solution for any graph model to incorporate generalizable structural information, reducing the need for domain-specific fine-tuning on heterogeneous web-scale tasks.

2 Related Work

Graph Pre-training. Graph self-supervised learning approaches are typically pre-training graph models, *e.g.*, GNNs or Graph Transformers, on a massive amount of labeled graphs with inherent features by reconstructing the structures or masked attributes [19, 38, 42, 47, 65, 83, 88, 89, 102]. Some works also utilize contrastive learning to enhance node and graph-level representation learning [31, 35, 36, 41, 51, 54, 59, 74, 75, 81, 90, 101]. These methods, while effective in certain domains, exhibit limited generalizability across different graph domains due to their tailored design for specific types of data. Additionally, there have been graph prompt techniques [28, 43] that can be used to enhance model adaptation over graphs. There have also been some attempts at cross-domain graph pre-training models [20, 70]. Unfortunately, all these models rely on one singular pre-training task (*i.e.*, contrastive learning),

and usually fail to capture fine-grained structural features at the node level or edge level [61].

Generalizable Positional and Structural Encoding (PSE). Early research on the web graph focused on characterizing its macroscopic properties, such as power-law degree distributions and the prevalent ‘bow-tie’ structure [2, 11, 26], which led to seminal node-importance algorithms like PageRank [10]. This perspective inspired a family of traditional, hand-crafted Positional and Structural Encodings (PSEs) based on such structural statistics, including Laplacian PE [4, 21, 50, 85], shortest-path distance [55, 96], random-walk encodings [12, 24, 60, 73], and node degree [96]. However, these global metrics and engineered features often fail to capture the complex, local connectivity patterns and subgraph motifs that signify sophisticated behaviors like spam link farms or coordinated user activity [52, 64]. To overcome these limitations, recent work has shifted towards models that learn PSE adaptively [14, 24, 50, 56]. While some methods like GPSE [58] pre-train a structural encoder, they still suffer from limited cross-domain transferability due to simplistic backbones. Developing an effective and versatile PSE remains an open challenge that requires further innovation.

Graph Language Models. With the success of foundation models in NLP, recent efforts also harness Large Language Models (LLMs) to develop domain-specific graph foundation models. This typically involves converting graph data into a text-compatible format by linearizing the graph’s structure (*e.g.*, as a sequence of adjacency lists or edge sets) and verbalizing node/edge attributes to create descriptive prompts [15–17, 27, 33, 49, 53, 57, 69, 78, 95, 100, 103]. The goal is to create a single, powerful model that can reason over both the semantics of text attributes and the graph’s topology. However, this approach faces a fundamental challenge: LLMs, which are pre-trained on sequential text, struggle to comprehend the complex, non-Euclidean nature of graph structures. Recent empirical studies and benchmarks consistently show that serializing a graph into a one-dimensional sequence leads to a significant loss of topological information, impairing an LLM’s ability to reason about complex connectivity, distances, and higher-order patterns [29, 82, 84]. Our work addresses this gap directly. Instead of relying on a single model for both text and structure, we focus on developing an expressive graph model dedicated solely to encoding rich topological information. Our approach is designed to be complementary to LLMs, serving as a universal structural encoder that can provide the crucial topological context that LLMs inherently lack, thereby enabling a more powerful and synergistic approach for text-attributed graphs.

3 Proposed Method

As shown in Figure 1, we collect graph pre-training datasets from six different domains, including social networks, product networks, academic networks, images, molecules, and proteins. GFSE utilizes a transformer-based architecture with biased attention to incorporate relative inductive bias within graph structures (Sec. 3.1). GFSE is pre-trained with four challenging self-supervision tasks simultaneously, each designed to enhance a crucial aspect of structural awareness and promote encoding quality (Sec. 3.3). GFSE generates expressive positional and structural encoding (PSE) for topological tasks (Sec. 3.2). The generated PSE can also be seamlessly integrated

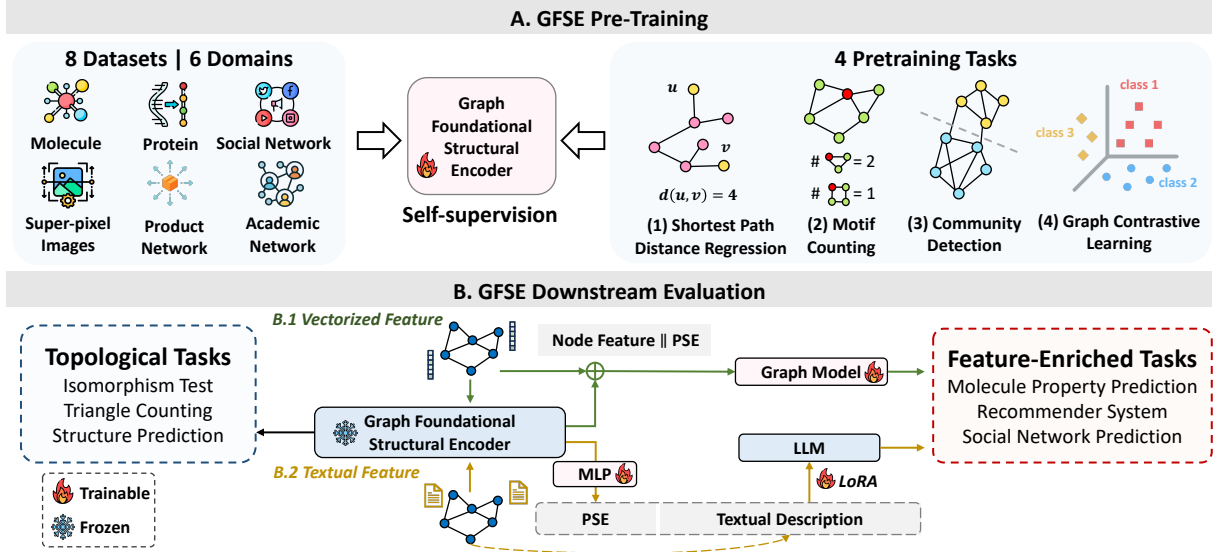


Figure 1: A) GFSE is pre-trained on 8 datasets from 6 different domains. Pre-training tasks include reconstruction and contrastive learning. B) GFSE generates generic and expressive Positional and Structural Encoding (PSE) to tackle topological graph tasks, which can also be seamlessly integrated into downstream feature encoders for feature-enriched tasks by concatenating with initial vectorized features or prepending the generated PSE to the textual prompt as a soft token.

into graphs with vectorized features or textual features, enhancing the downstream performance (Sec. 3.4).

3.1 Architecture

Previous work [58] uses randomized features to replace initial node features. However, it leads to poor generalizability across different domains. In this work, we propose to use both absolute and relative random-walk positional encoding as the initial features. Formally, let $G(V, E)$ represent an input graph, where V and E denote the set of nodes and edges, respectively. $A \in \mathbb{R}^{N \times N}$ indicates the adjacency matrix, where N is the number of nodes, and D is the degree matrix. Random Walk matrix is defined as $M = D^{-1}A$, where $M_{i,j}$ indicates the transition probability from the i -th node to the j -th node. Following previous works [60], we calculate the d -dimensional encoding for each node and node pairs as follows.

$$P_i = [I, M, M^2, \dots, M^{d-1}]_{i,:}, \quad R_{i,j} = [I, M, M^2, \dots, M^{d-1}]_{i,j} \quad (1)$$

$P \in \mathbb{R}^{N \times d}$ and $R \in \mathbb{R}^{N \times N \times d}$ are used as the initial node features and edge features in GFSE.

Pre-training Backbone. GFSE is built on a GPS architecture [73] for pre-training due to its scalability and generalizability. Each GPS layer contains local message passing and global attention modules to capture neighbor and long-range information. In the ℓ -th layer, the node encoding P^ℓ and relative edge encoding R^ℓ are fed into message passing layers (MPNN) and Biased Attention Module (BiasAttn) parallelly. The node encoding is updated by aggregating local and global information $P^{\ell+1} = \text{MLP}^\ell(P_M^{\ell+1} + P_T^{\ell+1})$ where

$$P_M^{\ell+1}, R^{\ell+1} = \text{MPNN}^\ell(P^\ell, R^\ell, A); \quad P_T^{\ell+1} = \text{BiasAttn}^\ell(P^\ell, R^\ell). \quad (2)$$

Attention Bias. The global attention in the original GPS framework does not account for relative edge encoding while leaving

them entirely for the message-passing layers. However, incorporating relative edge encoding in global attention is crucial for capturing long-range dependencies, as the receptive field of message-passing layers is inherently constrained by their depth. To build a theoretically more powerful encoder, we explicitly incorporate relative edge encoding into global attention, where the attention weight between the i -th and the j -th nodes is computed by $a'_{i,j} = \text{SoftMax}(a_{i,j} + \text{Linear}(R_{i,j}))$, where $\text{Linear} : \mathbb{R}^d \rightarrow \mathbb{R}$ indicates a linear layer that maps the d -dimensional relative encoding to a scalar. $a_{i,j}$ denotes the original attention weight computed by scaled-dot self-attention on the node encoding P^ℓ in each GPS layer.

3.2 Expressive Power of GFSE

We show that GFSE can generate highly expressive PSE by incorporating relative edge encoding into the attention computation. To characterize the expressiveness of GFSE, we employ the Structural Encoding enhanced Global Weisfeiler-Lehman test (SEG-WL) [105], a generalized WL test that incorporates relative structural encoding into the isomorphism algorithm. For an input graph $G(V, E)$ with node set V and edge set E , let $f_P : V \rightarrow \mathcal{X}$ and $f_R : V \times V \rightarrow \mathcal{X}$ indicate the node-level and edge-level structural encoding, respectively. Different from traditional WL test, SEG-WL updates the node labels at the t -th iteration by $g_t(v) = \text{hash}(\{(g_{t-1}(u), f_R(v, u)) : u \in V\})$ and $g_0(v) = \text{hash}(f_P(v))$. SEG-WL can be viewed as a high-level abstraction of the learning paradigm of our pre-training architecture with biased attention (Eq. 2), where relative structural encoding between any two nodes is considered for updating node representations [105]. Let RW(d)-SEG-WL denote the case that f_P and f_R are determined by P and R with d dimension, i.e., $f_P(v_i) = P_i \in \mathbb{R}^d$ and $f_R(v_i, v_j) = R_{ij} \in \mathbb{R}^d$ for the i -th and j -th nodes. We have the following propositions.

Proposition 3.1. *RW(d)-SEG-WL ($d \geq 3$) is strictly more expressive than 1-WL in testing non-isomorphic graphs.*

PROOF. We first introduce Neighbor-SEG-WL, which is the SEG-WL test when f_p is an identity encoding and $f_R(u, v)$ equals 1 if $(u, v) \in E$ and 2 otherwise. Previous works have proved the following Proposition [105]:

LEMMA 1. *Two non-isomorphic graphs can be distinguished by WL if and only if they are distinguishable by Neighbor-SEG-WL.*

Therefore, Neighbor-SEG-WL is a specific example of SEG-WL test that has equivalent expressiveness to the 1-WL test. We then prove that RW-SEG-WL is strictly more expressive than Neighbor-SEG-WL. Let $d_{\text{neg}}(u, v)$ indicate the edge-level encoding f_R in Neighbor-SEG-WL. Note $d_{\text{neg}}(v_i, v_j) = 2$ if and only if $A_{ij} = 0$. Recall that $f_R(\cdot, \cdot)$ in RW-SEG-WL satisfies $f_R(v_i, v_j) = \mathbf{R}_{ij} \in \mathbb{R}^d$ with $\mathbf{R} = [\mathbf{I}, \mathbf{M}, \dots, \mathbf{M}^{d-1}]$ where $\mathbf{M} = \mathbf{D}^{-1}\mathbf{A}$. Therefore, f_R in RW-SEG-WL strictly contains the information of d_{neg} . Therefore, if two non-isomorphic graphs can be distinguished by WL, they can be distinguished by RW-SEG-WL. Proposition 3.1 is proved. $\square$

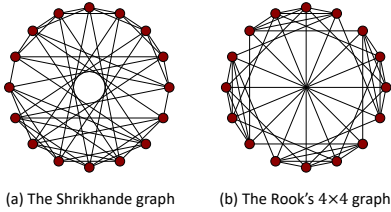


Figure 2: RW(d)-SEG-WL can distinguish two non-isomorphic graphs when $d > 4$ while 3-WL fails.

Proposition 3.2. *There exist pairs of graphs that RW(d)-SEG-WL can distinguish, but 3-WL can not.*

PROOF. To prove Proposition 3.2, we provide an example in Figure 2 which shows the Shrikhande graph and the Rook's 4×4 graph, a pair of strongly regular graphs. It has been proved that they cannot be distinguished by 3-WL [3]. We empirically verified that RW(d)-SEG-WL with $d > 4$ can distinguish these two graphs. $\square$

Synthetic Graph Isomorphism Tests. To evaluate the expressive power of RW-SEG-WL, we perform synthetic graph isomorphism tests on low-order graphs and strongly regular graphs. We consider low-order graphs with up to 8 nodes. Strongly regular graph pair $\text{SRG}(n, k, \lambda, \mu)$ means graphs with n nodes, where each node has k neighbors. Each adjacent pair of nodes has the same number λ of neighbors in common, and each non-adjacent node pair has μ neighbors in common. Strongly regular graphs are known to be challenging cases for graph isomorphism test algorithms due to their highly symmetric structure [3]. We compare with 1-WL and SPD-SEG-WL, where f_p is an identity encoding and f_R is defined as the shortest path distance between two nodes. Note SPE-SEG-WL can be viewed as an expressivity upper bound of Graphormer [96]. The results are shown in Table 1. We observe that RW-SEG-WL can distinguish significantly more non-isomorphic graphs than 1-WL and SPD-SEG-WL. Specifically, with d equals 8, i.e., when considering a random walk with 8 steps, RW-SEG-WL successfully

distinguishes all low-order graphs and strongly regular graphs. When setting $d = 4$, there are 16 pairs of strongly regular graphs that cannot be distinguished by RW-SEG-WL. Therefore, it is natural to develop a graph transformer equipped with a relative random-walk encoding that can accurately capture important graph structures and demonstrate strong expressive power.

RW-SEG-WL stands as an expressivity upper bound of our proposed GFSE. The pre-training tasks are meticulously designed to push GFSE towards achieving the upper bound established by RW-SEG-WL. The pre-training tasks are carefully designed to optimize both node-level and edge-level structural encoding, progressively refining the effectiveness of the model in generating expressive encoding.

3.3 Self-supervised Pre-training Tasks

GFSE is pre-trained with reconstruction and contrastive structural tasks at multiple context levels. Each task highlights a specific structural aspect, thereby augmenting the model's expressiveness and capability to capture complex graph structures. Let $\mathbf{P}^L \in \mathbb{R}^{N \times d_e}$ represent the output after L layers. We decode $\mathbf{P}^L$ with independent MLP heads for each pre-training task.

Shortest Path Distance Regression encodes the global proximity and connectivity between nodes, which helps to discern nodes' positions and relations within the entire graph [55]. We pre-compute the shortest path distance via the Dijkstra algorithm [22] to create the label $\text{SPD} \in \mathbb{R}^{N \times N}$. The loss for shortest path distance regression is computed by $\mathcal{L}_{\text{SPD}} = \frac{1}{|E|} \sum_{i,j \in V} \|\mathbf{h}_{\text{SPD}}(\mathbf{P}_i^L \parallel \mathbf{P}_j^L) - \text{SPD}_{i,j}\|^2$, where $\parallel$ indicates the concatenation and $\mathbf{h}_{\text{SPD}}$ indicates a task-specific head.

Motif Counting enables the model to better identify each node's role in the surrounding subgraphs. To improve expressiveness, we include a variety of small motifs, called *graphlets* [8, 67, 68], with different numbers of vertices, which are beyond usual types like stars, paths, cycles, and cliques. Let $Y_i^m \in \mathbb{Z}^k$ denote the motif label that indicates the number of certain motifs surrounding node i , where k is the number of graphlet types. The loss is formulated as $\mathcal{L}_{\text{MC}} = \frac{1}{|V|} \sum_{i \in V} \|h_{\text{MC}}(\mathbf{P}_i^L) - Y_i^m\|^2$, where $h_{\text{MC}}: \mathbb{R}_e^d \rightarrow \mathbb{R}^k$ is the task-specific head for motif counting.

Community Detection aims to identify densely connected subgraphs, where nodes within a community are more closely linked to each other than to nodes outside the community. Such community structures are ubiquitous in various real-world networks, e.g., social networks, and transportation systems. We employ the Louvain Community Detection Algorithm [6] to extract the community structure from pre-training graphs, which clusters nodes into communities based solely on graph topology without node features. We approach this task in a contrastive learning manner by minimizing the embedding distances between intra-community nodes while maximizing the distance between inter-community nodes.

$\mathcal{L}_{\text{CD}} = \sum_{i \in V} \sum_{j \in V} Y_{i,j}^c (1 - \text{sim}(i, j)) + (1 - Y_{i,j}^c) \max(0, \epsilon - (1 - \text{sim}(i, j)))$, where the similarity score $\text{sim}(i, j)$ is calculated by $\text{sim}(i, j) = \frac{\mathbf{z}_i \cdot \mathbf{z}_j}{\|\mathbf{z}_i\| \cdot \|\mathbf{z}_j\|}$ and $\mathbf{z}_i = h_{\text{CD}}(\mathbf{P}_i^L)$ with a head h_{CD} . ϵ is a margin hyperparameter. $Y_{i,j}^c$ is a binary label that indicates if the i -th node and the j -th node are in the same community. Through $\mathcal{L}_{\text{CD}}$, GFSE

Table 1: Results of synthetic graph isomorphism tests

	Low-Order Graphs (Parameter: n)				Strongly Regular Graphs (Parameter: (n, k, λ, μ))					
Parameter	5	6	7	8	(25,12,5,6)	(26,10,3,4)	(29,14,6,7)	(36,14,4,6)	(40,12,2,4)	45,12,3,3)
# graphs	21	112	853	11117	15	10	41	180	28	78
# graph pairs	210	6216	363378	61788286	105	45	820	16110	378	3003
number of undistinguishable graph pairs										
WL	0	3	17	312	105	45	820	16110	378	3003
SPD-SEG-WL	0	2	12	186	105	45	820	16110	378	3003
RW-SEG-WL($d=8$)	0	0	0	0	0	0	0	0	0	0

learns to discern community boundaries and distinguish nodes based on local community structures.

Graph Contrastive Learning aims to distinguish graphs from different datasets. The motivation is rooted in the observation that similar structural characteristics in different domains may exhibit distinct meanings. For example, a densely connected subgraph in a citation network might correspond to a specific research topic, whereas a similar structure in a social network could represent a tightly-knit community or interest group. Therefore, GFSE distinguishes graphs from different datasets with the contrastive loss $\mathcal{L}_{GCL} = -\log \frac{\exp(\text{sim}(z_{G_i}, z_{G_j})/\tau)}{\sum_{k=1}^K \mathbb{1}_{[G_k \sim G_i]} \exp(\text{sim}(z_{G_i}, z_{G_k})/\tau)}$, where τ is the temperature, G_i and G_j are from the same dataset, $z_{G_i} = \text{GlobalPool}(h_{GCL}(\mathbf{P}_{G_i}^L))$ is the output of the global pooling applied to the last layer’s representation $\mathbf{P}_{G_i}^L$ for the graph G_i , K is the number of negative samples, and $\mathbb{1}_{[G_k \sim G_i]}$ indicates whether graphs G_k and G_i originate from different datasets.

Multi-task Selection and Loss Balance: Each pre-training task targets a different structural aspect, enabling GFSE to capture a comprehensive understanding of graph topology. For instance, the shortest path distance regression task emphasizes global connectivity within graphs, while motif counting focuses on the occurrence of local subgraph patterns. Although other pre-training tasks could be considered, the chosen tasks provide a balanced and effective combination, capable of capturing both local and global structural information at varying granularities without introducing unnecessary complexity. Since the loss scale and difficulty vary significantly across tasks, we employ task-specific uncertainty [45] to unify the loss scales, which automatically balances different pre-training losses, *i.e.*, $\mathcal{L}_{SPD}$, $\mathcal{L}_{MC}$, $\mathcal{L}_{CD}$, and $\mathcal{L}_{GCL}$ (see Appendix A.3 for more details). Moreover, tracking the evolution of uncertainty values offers an interpretable perspective on how each task contributes to the overall pre-training process, enabling a deeper understanding of the model’s learning dynamics and task interactions.

3.4 Combination with Downstream Feature Encoder

Application on Graphs with Vectorized Features. GFSE can be readily employed to generate expressive PSE for various graph applications. Let $\mathbf{X}^0 \in \mathbb{R}^{N \times d_x}$ denote the initial node features for a given graph with N nodes and $\mathbf{P}^L \in \mathbb{R}^{N \times d_e}$ denote PSE generated by GFSE, where d_x and d_e are dimensions of node features and PSE, respectively. $\mathbf{P}^L$ can then be concatenated with the initial node features $\mathbf{X}^0$ to create a new feature matrix $\mathbf{X}^{new} = [\mathbf{X}^0 || \mathbf{P}^L] \in$

$\mathbb{R}^{N \times (d_x + d_e)}$, which augments the node features with structural information. This structure-enriched feature $\mathbf{X}^{new}$ can subsequently be fed into downstream graph models, such as graph neural networks or graph transformers, enhancing their performance on various tasks. For large-scale graphs, where computing PSE for the entire graph may be computationally prohibitive, we thereby sample the neighborhood structure around each node and compute the PSE for these localized subgraphs, which is efficiently parallelized, enabling scalable and efficient generation of PSE for large graphs.

Application on Text-attributed Graphs. Text-attributed graphs consist of nodes associated with textual descriptions and edges representing their relations. Language models are commonly employed to capture both textual and structural information. In this setting, GFSE can be seamlessly integrated to inject structural awareness into textual representations. Given the generated $\mathbf{P}^L \in \mathbb{R}^{N \times d_e}$, a lightweight MLP is trained to project $\mathbf{P}^L$ into the embedding space of the language model. The projected PSEs are prepended as soft tokens to the textual inputs, thereby enriching them with graph topology cues. These structure-augmented tokens are then processed by downstream LLMs, improving their understanding and reasoning on graph-related tasks. The approach, involving training a lightweight MLP and fine-tuning LLM with Parameter-Efficient Fine-Tuning (PEFT) techniques such as LoRA [39], makes it scalable and efficient for large-scale text-attributed graph applications.

3.5 Computational Complexity

GFSE’s complexity comprises two parts: pre-computation of self-supervision labels and pre-training. Computing the shortest path distance between node pairs results in the time complexity of $O(|E| + |V| \log |V|)$ with node set V and edge set E . A brute-force implementation of the subgraph isomorphism counting of fixed size t is $O(|V|^t)$. We consider the graphlets with at most 5 nodes. One can also choose special graphlet types, *e.g.*, paths, cycles, and triangles, which can be efficiently enumerated [32]. Approximating and scalable algorithms can be further used to accelerate this pre-processing step [30, 66, 97]. For pre-training, the complexity is $O(|V|^2)$ for full attention computation and $O(d|V||E|)$ for initial encoding computation. See runtime evaluations in Appendix B.3.

4 Experiments

We first evaluate the pre-training performance in Sec. 4.2 and empirically assess the expressiveness of GFSE on synthetic datasets in Sec. 4.3. We then evaluate GFSE in a wide range of downstream graph learning tasks in Sec. 4.4. Specifically, we conduct experiments with LLMs on text-attributed graphs in Sec. 4.5.

4.1 Pre-Training Setup

Dataset. We utilize diverse datasets for pretraining, ensuring a broad spectrum of graph structures and scales, including MolPCBA, MolHIV, MNIST, peptides, ogbn-proteins, Pokec, ogbn-arxiv and ogbn-product [5, 18, 23, 25, 63, 76, 77, 86]. These datasets cover several real-world graph domains, such as social networks, academic networks, *etc.* We refer to Table 8 in Appendix A.1 for detailed dataset statistics. For large-scale graphs, we first partition them into sets of subgraphs by the METIS algorithm [44] to handle scalability issues. Training samples from different datasets are mixed and randomly shuffled to form a large-scale pre-training dataset.

Pre-training Setting. The pre-training stage is conducted on the standard train/validation/test splits of the pre-training datasets. The initial encoding dimension d is set as 8. We adopt GIN [91] as the message-passing layer in the GPS and adopt 8 GPS layers with 8 heads and 128 hidden dimensions. The output dimension is 64 by default. We use Adam as the optimizer with an initial learning rate of 0.001 and the batch size is 256. The maximum training epochs is 100. An early stopping strategy is used to mitigate overfitting. The pre-training is implemented on the NVIDIA A40 48GB GPU. We refer to Appendix A.4 for more details.

4.2 Pre-training Evaluation

GFSE is pre-trained with four self-supervised learning tasks. We iteratively change the message passing layers (*e.g.*, GatedGCN [9], GCN [46] and GIN [91]) and replace the biased attention with traditional self-attention in the default GFSE architecture. We evaluate the pre-training performance with different architectures as shown in Figure 3. Accuracy is used to measure community detection and graph contrastive learning tasks, indicating the proportion of node (graph) pairs that are correctly predicted. MSE and MAE are used for shortest path distance and motif counting tasks. See more details in Appendix A.2. We observe a consistent performance boost when combining biased attention with GIN, which serves as the default configuration of GFSE.

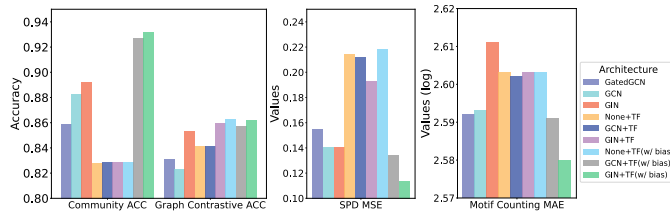


Figure 3: Pre-training performance with different architectures. TF stands for transformer.

4.3 Expressiveness Evaluation

We empirically evaluate the structure-awareness of the positional and structural encoding (PSE) generated by GFSE on Triangle [48], Pattern, and Cluster [23], three datasets that require discerning intricate graph topologies. We evaluate the performance boost brought by GFSE in comparison to traditional RWSE and LapPE. We test on various downstream graph learning models, including MLP, GIN [91], transformer [80] and GPS [73]. We further compare with the learning-based PSE approach: GPSE [58]. As shown in

Table 2: Test accuracy (%) enhanced by different positional and structural encoding. The results are averaged over five random seeds. The best results in each dataset are bolded.

	Triangle-S	Triangle-L	Pattern	Cluster
MLP+RWSE	98.22	11.88	50.53	20.96
MLP+LapPE	98.60	12.62	50.53	20.96
MLP+GPSE	52.80	17.42	55.66	20.96
MLP+GFSE	98.71	25.54	57.79	21.28
GIN	99.68	42.58	85.58	60.84
GIN+RWSE	99.70	40.78	85.34	61.30
GIN+LapPE	99.74	42.48	85.45	61.83
GIN+GPSE	99.32	25.32	85.19	61.95
GIN+GFSE	99.72	43.84	85.58	63.49
Transformer (TF) for Triangle and GPS for Others				
TF / GPS	21.68	23.58	86.63	77.76
TF / GPS+RWSE	35.96	11.38	86.68	77.72
TF / GPS+LapPE	35.96	12.44	86.54	77.76
TF / GPS+GPSE	62.04	24.64	85.58	77.80
TF / GPS+GFSE	92.82	30.15	87.98	77.86

Table 2, GFSE generates expressive and robust PSE that consistently improves the base model’s performance, whereas other structural encodings exhibit considerable variation across different datasets or different base models. Notably, the performance boost brought by GFSE is particularly significant for the Transformer on the Triangle dataset. This demonstrates that GFSE has a stronger enhancement effect on models that originally lack structural bias.

4.4 Downstream Evaluation

4.4.1 Evaluation on Pre-training Domains. We conduct a comprehensive evaluation of GFSE on real-world graph datasets. We adhere to the experimental setting and hyper-parameters established by previous works [73] to implement base models. We augment the initial node features with GFSE and evaluate the performance in downstream tasks, comparing it against established structural encoding methods including RWSE, LapPE, and GPSE [58]. Evaluation results on citation networks (*e.g.*, ogbn-Arxiv, PubMed), and image graphs (*e.g.*, MNIST and CIFAR10) are given in Table 3, where PubMed and CIFAR10 are out of the pre-training dataset. More results are provided in Table 10 in Appendix B.2.

Table 3: Test Accuracy (%) on ogbn-Arxiv, PubMed, MNIST and CIFAR10.

	ogbn-arxiv	PubMed	MNIST	CIFAR10
GateGCN	71.69±0.21	76.86±0.41	97.34±0.14	67.31±0.31
GateGCN+LapPE	71.95±0.37	74.83±0.24	97.10±0.28	65.08±0.26
GateGCN+RWSE	71.83±0.65	76.11±0.39	96.84±0.27	65.26±0.68
GateGCN+GPSE	72.17±0.42	71.97±0.36	96.94±0.17	65.63±0.27
GateGCN+GFSE	72.61±0.53	78.39±0.84	97.44±0.31	68.39±0.47
Transformer (TF)	5.86±0.00	66.63±0.73	97.29±0.11	69.04±0.28
TF+LapPE	5.86±0.00	66.27±0.46	96.95±0.38	69.01±0.61
TF+RWSE	5.86±0.00	64.43±0.37	97.81±0.58	70.70±0.45
TF+GPSE	21.56±2.74	65.89±0.14	97.78±0.32	69.57±0.16
TF+GFSE	23.84±3.15	66.30±0.68	98.03±0.84	71.33±0.23
GPS	70.68±0.71	74.26±0.60	98.05±0.12	71.49±0.35
GPS+LapPE	69.51±0.38	73.68±0.37	98.16±0.28	71.87±0.21
GPS+RWSE	72.14±0.84	72.87±0.44	98.19±0.30	71.30±0.33
GPS+GPSE	71.21±0.34	73.71±0.70	98.08±0.13	72.31±0.25
GPS+GFSE	72.30±0.13	75.13±0.35	98.15±0.46	74.11±0.93

Results. From Table 3, we observe that the integration of GFSE achieves state-of-the-art performance in 10 out of 12 evaluated cases. This enhancement is not confined to a specific domain; it is evident across diverse tasks, including node classification on large-scale citation networks and graph-based image classification. Notably, GFSE consistently outperforms both traditional hand-crafted encodings, which show inconsistent benefits, and the existing pre-trained baseline, GPSE. The dramatic performance leap observed when applying GFSE to the standard Transformer is particularly telling, as it highlights our model’s ability to inject essential structural inductive biases into architecturally general models, enabling them to effectively reason over complex graph data.

4.4.2 Evaluation on Molecular Graphs. We use the molecular datasets, namely Tox21, Sider, BBBP, and MUV [40] as a downstream task. We evaluate the effectiveness of GFSE under two settings: (1) In the training-from-scratch setting, we directly concatenate GFSE’s PSE with the raw node features to create new input features. This augmented representation is then fed into a randomly initialized model from the beginning of training. We take GINE [91] and GPS [73] as our backbone. (2) In the fine-tuning setting, we assess GFSE’s ability to enhance pre-trained models by concatenating the node encodings obtained from a pre-trained model with the GFSE’s PSE. The concatenated features are then fed into the final read-out layers for prediction. During fine-tuning, the parameters of the pre-trained model and read-out layers are updated. We select GraphMAE [38] and MoleBERT [87] as the pre-trained backbones and compare with other baselines without structural encoding, namely SSP [41], GraphLoG [92], GraphCL [98]. Refer to Appendix B.1 for more implementation details.

Results. Experimental results are shown in Table. 4. For training the models from scratch, on both GINE and GPS, PSE consistently improves model performance, achieving better results than all the other structural feature augmentation methods. As to fine-tuning, GFSE significantly boosts the performance and achieves state-of-the-art performance.

Table 4: Test ROC-AUC(%) performance on molecular datasets. The best results for each dataset are bolded.

	Tox21	Sider	BBBP	MUV
SSP	76.8 ± 0.8	61.7 ± 0.8	67.9 ± 0.9	79.8 ± 1.6
GraphCL	75.7 ± 0.5	60.8 ± 0.7	69.5 ± 0.5	74.5 ± 1.3
GraphLoG	75.4 ± 0.9	61.2 ± 1.1	72.5 ± 0.8	76.0 ± 1.1
Train-from-scratch Models				
GINE	74.5 ± 0.4	58.6 ± 0.1	67.7 ± 0.7	74.8 ± 0.6
GINE+RWSE	75.3 ± 0.2	58.4 ± 1.8	66.7 ± 1.4	76.4 ± 0.8
GINE+LapPE	77.6 ± 0.8	57.2 ± 1.1	65.8 ± 0.3	77.0 ± 0.8
GINE+GPSE	74.9 ± 0.4	60.1 ± 0.8	66.4 ± 0.1	75.8 ± 1.3
GINE+GFSE	75.5 ± 0.7	60.9 ± 0.5	69.1 ± 1.3	77.7 ± 1.2
GPS	73.9 ± 0.2	58.6 ± 0.4	67.1 ± 0.3	68.0 ± 0.6
GPS+RWSE	74.6 ± 1.3	56.4 ± 0.6	67.9 ± 1.0	69.7 ± 0.6
GPS+LapPE	74.8 ± 1.1	60.5 ± 0.6	67.9 ± 0.6	70.1 ± 2.2
GPS+GPSE	75.1 ± 0.7	56.6 ± 1.7	67.8 ± 0.7	68.3 ± 0.1
GPS+GFSE	76.3 ± 1.4	61.8 ± 0.5	68.0 ± 0.5	73.6 ± 0.5
Pre-trained Models				
GraphMAE	75.4 ± 0.4	59.8 ± 0.5	69.5 ± 1.6	76.3 ± 2.4
GraphMAE+RWSE	76.3 ± 0.5	60.5 ± 0.8	66.4 ± 3.7	77.7 ± 1.5
GraphMAE+GFSE	75.9 ± 0.9	62.1 ± 0.8	70.5 ± 1.4	78.1 ± 1.3
MoleBERT	76.8 ± 0.5	62.8 ± 1.1	71.9 ± 1.6	78.6 ± 1.8
MoleBERT+RWSE	77.8 ± 0.7	63.1 ± 0.6	66.5 ± 2.1	80.4 ± 1.3
MoleBERT+GFSE	78.0 ± 0.4	63.1 ± 0.7	68.9 ± 2.1	80.5 ± 2.0

Table 5: Evaluation on text-attributed graphs

	Cloth		Home		Sport		Arxiv	Cora
	Hit@1	MRR	Hit@1	MRR	Hit@1	MRR	ACC	ACC
InstructGLM	76.23	82.60	79.82	85.93	62.50	73.25	75.70	87.08
Finetuned LLaMA	74.73	82.87	78.93	86.07	62.52	75.77	74.94	79.95
+ RRWP	76.08	84.11	74.64	85.24	62.71	75.20	76.03	90.27
+ GraphSAGE	76.22	84.16	73.74	81.66	62.26	75.36	76.13	90.83
+ GFSE (Ours)	76.84	84.68	79.85	86.77	64.79	76.24	76.10	91.26

4.5 Integration with Large Language Models

We evaluate GFSE on the task of link prediction using three text-attributed graphs: the citation networks Arxiv [40] and Cora [94], and the e-commerce network Amazon-Product [37, 62]. In these datasets, each node (e.g., a paper or product) features a detailed textual description, and edges signify relations such as co-authorship or co-purchase. To synergize our structural encodings with an LLM, we employ a lightweight MLP to project the 32-dimensional PSE into the 4096-dimensional embedding space of LLaMA2-7B [79], aligning the structural and textual modalities. The input for the LLM is constructed by concatenating the central node’s text with that of its one-hop neighbors. This sequence is then prepended with the projected PSE as a soft structural token and appended with a special graph token. The final hidden state corresponding to this special token serves as the comprehensive node representation. Following a prior setup [104], we train the model to predict edge likelihoods based on the cosine similarity between node-pair representations. Specifically, we train the MLP projector and fine-tune the LLM using LoRA [39] with the contrastive objective: $\mathcal{L} = \sum_{(i,j) \in E \& (i,j') \notin E} (d_{ij}^2 + \max(\tau - d_{ij'}, 0)^2)$, where $d_{ij} = 1 - \cos(v_i, v_j)$, v_i denotes the i -th node representation, and $\tau = 0.5$ is the margin.

Results. Evaluation results on text-attributed graphs are reported in Table 5. We select InstructGLM [95] as a baseline, which has been fine-tuned on graph domains without structural information. Additionally, we include comparisons against RRWP [60], and GraphSAGE [34], which are trained from scratch as a PSE encoder with LLM finetuning together. *Finetuned LLaMA* refers to the LLaMA model [79] fine-tuned without any PSE. As seen from the table, GFSE, which is pre-trained on cross-domain graph data as a structural encoder, outperforms other methods in general. In particular, GFSE achieves an average performance boost of 2.66% over InstructGLM, 2.02% over GraphSAGE encoder and 1.53 over RRWP across the five datasets. These gains demonstrate the effectiveness of GFSE in language-based graph tasks. See more discussions in Appendix B.2.

4.6 More Experimental Analysis

Reconstruction of other PSE types. The pre-trained GFSE is able to generate effective PSE, which can reconstruct other pre-defined PSE, such as LapPE [56] and ElstaticPE [50]. Table 6 demonstrates that the PSE generated by the pre-trained GFSE, followed by a trainable lightweight MLP, is capable of reconstructing various other pre-defined PSEs. We evaluate this using the coefficient of determination R^2 scores as a metric. Notably, our model performs competitively compared with GPSE, given the fact that GPSE directly adopts PSE reconstruction as its training objective. Instead,

our method generalizes well across different PSEs without being directly trained for reconstruction. This suggests that the structural self-supervision pre-training tasks are effective and sufficient in capturing important structural information.

Table 6: Performance of other PSE reconstruction. The coefficient of determination R^2 scores are reported as the metric.

PSE type	ElasticPE	LapPE	RWSE	HKdiagSE	CycleSE
GPSE	0.964	0.973	0.984	0.981	0.977
Ours	0.947	0.986	0.987	0.984	0.992

Ablation on Tasks and Model Architecture. We analyze the sensitivity of each pre-training task and model architecture *w.r.t.* the performance boost in downstream tasks. The results are illustrated in Table 7. Firstly, we iteratively remove one pre-training task and follow the same setting to pre-train and evaluate GFSE. We observe that each removal results in a discernible reduction in downstream performance. Notably, the shortest path distance task is particularly critical for the ZINC and CIFAR10 datasets, while community detection is significant for academic datasets. We also conduct ablation studies on the main components of GFSE. Specifically, we use traditional attention to replace biased attention and remove GIN or attention modules, respectively. We notice that all the above architecture variants lead to performance degradation. The hybrid approach outperforms both attention-only and GIN-only setups, suggesting that integrating biased attention mechanisms can compensate for the absence of global information in local message-passing layers.

Table 7: Ablation studies on the pre-training tasks and model architecture of GFSE. Best results are shown in bold.

	ZINC MAE ↓	CIFAR10 ACC ↑	Arxiv ACC ↑
GPS w/o PE	0.1182	71.49	70.68
Augment by GFSE	0.0613	74.11	72.30
Different Pre-training Tasks			
w/o Community Detection	0.0637	72.38	70.34
w/o Motif Counting	0.0731	73.02	71.27
w/o Shortest Path Distance Regression	0.1074	71.58	72.06
w/o Graph Contrastive Learning	0.0856	73.02	72.13
Different Model Architecture			
GIN+Traditional Attention	0.0872	73.13	71.85
Biased Attention Only	0.1137	70.97	71.33
GIN Only	0.0640	72.31	72.34

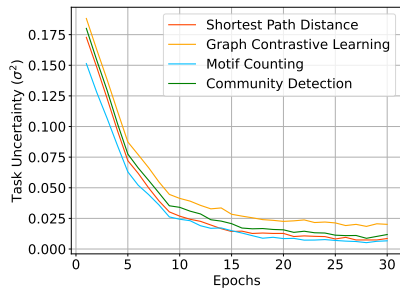


Figure 4: Learning task uncertainty (σ^2) *w.r.t.* pre-training

Task-specific Uncertainty. Figure 4 illustrates the trajectory of task uncertainty (σ^2) across different pre-training tasks *w.r.t.* pre-training epochs. Higher values of σ^2 reduce the respective task's contribution to the overall training loss. We observe that all tasks

show a sharp decline in uncertainty during the pre-training process. Notably, motif counting maintains a lower uncertainty throughout the training process compared to other tasks, suggesting that it might be inherently more straightforward for the model to optimize or more integral to the model's overall learning structure.

Impact of Pre-training Data Size. Figure 5 illustrates the impact of pre-training data scale and domain transfer on GFSE's performance, revealing several key insights. First, as the pre-training data ratio increases, the model's performance consistently improves, indicating that larger and more diverse pre-training datasets are essential for capturing richer structural representations and achieving stronger generalization. Second, models initialized with GFSE pre-training outperform their train-from-scratch counterparts, demonstrating the effectiveness of structural pre-training in providing transferable inductive biases that enhance downstream adaptation. Lastly, cross-domain pre-training proves to be important for domain generalization. GFSE pre-trained on a single dataset struggles to generalize to different domains (*e.g.*, from MolPCBA to Arxiv). This indicates that models trained on a single domain may overfit to the specific characteristics, limiting their generalization to other tasks. In contrast, cross-domain pre-training enables the model to learn more robust, domain-invariant representations, crucial for scaling to diverse real-world text-attributed graph applications.

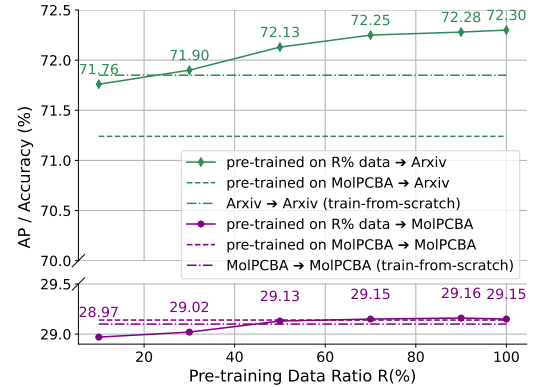


Figure 5: Scalability of GFSE with GPS as a backbone.

5 Conclusion

GFSE represents a meaningful step toward a universal graph encoder, leveraging multiple self-supervised objectives and relative positional encoding within a Graph Transformer to capture expressive and transferable structural patterns across diverse graph domains. Extensive experiments on both synthetic and real-world datasets demonstrate that GFSE consistently enhances the performance of downstream graph encoders across a wide range of tasks. The effectiveness of GFSE can be influenced by the quality and diversity of the pre-training graphs, as biases or under-representation in these data sources may affect the learned representations. Incorporating richer, more heterogeneous, and potentially dynamic graph data is a natural direction for future research. More broadly, GFSE holds promise for real-world applications such as computational biology, social network analysis, and recommendation systems, while reducing reliance on heavy task-specific fine-tuning and improving the accessibility of strong graph representations under limited computational budgets.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Alvenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [2] Réka Albert, Hawoong Jeong, and Albert-László Barabási. 1999. Internet: Diameter of the world-wide web. *Nature* 401, 6749 (1999), 130–131.
- [3] Vikraman Arvind, Frank Fuhlbrück, Johannes Köbler, and Oleg Verbitsky. 2020. On weisfeiler-leman invariance: Subgraph counts and related graph properties. *J. Comput. System Sci.* 113 (2020), 42–59.
- [4] Dominique Beaini, Saro Passaro, Vincent Létourneau, Will Hamilton, Gabriele Corso, and Pietro Liò. 2021. Directional graph networks. In *International Conference on Machine Learning*. PMLR, 748–758.
- [5] K. Bhatia, K. Dahiya, H. Jain, P. Kar, A. Mittal, Y. Prabhu, and M. Varma. 2016. The extreme classification repository: Multi-label datasets and code. <http://manikvarma.org/downloads/XC/XMLRepository.html>
- [6] Vincent D Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. 2008. Fast unfolding of communities in large networks. *Journal of statistical mechanics: theory and experiment* 2008, 10 (2008), P10008.
- [7] Zolán Borsos, Raphaël Marinier, Damien Vincent, Eugene Kharitonov, Olivier Pietquin, Matt Sharifi, Dominik Roblek, Olivier Teboul, David Grangier, Marco Tagliasacchi, et al. 2023. Audiollm: a language modeling approach to audio generation. *IEEE/ACM Transactions on Audio, Speech, and Language Processing* (2023).
- [8] Giorgos Bouritsas, Fabrizio Frasca, Stefanos Zafeiriou, and Michael M Bronstein. 2022. Improving graph neural network expressivity via subgraph isomorphism counting. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45, 1 (2022), 657–668.
- [9] Xavier Bresson and Thomas Laurent. 2017. Residual gated graph convnets. *arXiv preprint arXiv:1711.07553* (2017).
- [10] Sergey Brin and Lawrence Page. 1998. The anatomy of a large-scale hypertextual Web search engine. In *Proceedings of the Seventh International Conference on World Wide Web*. Elsevier, 107–117.
- [11] Andrei Broder, Ravi Kumar, Farzin Maghoul, Prabhakar Raghavan, Sridhar Rajagopalan, Raymie Stata, Andrew Tomkins, and Janet Wiener. 2000. Graph structure in the Web. In *Proceedings of the 9th international World Wide Web conference on Computer Networks*. 309–320.
- [12] Rickard Brül-Gabrielsson, Mikhail Yurochkin, and Justin Solomon. 2022. Rewiring with positional encodings for graph neural networks. *arXiv preprint arXiv:2201.12674* (2022).
- [13] Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott Lundberg, et al. 2023. Sparks of artificial general intelligence: Early experiments with gpt-4. *arXiv preprint arXiv:2303.12712* (2023).
- [14] Dexiong Chen, Leslie O’Bray, and Karsten Borgwardt. 2022. Structure-aware transformer for graph representation learning. In *International Conference on Machine Learning*. PMLR, 3469–3489.
- [15] Runjin Chen, Tong Zhao, Ajay Jaiswal, Neil Shah, and Zhangyang Wang. 2024. Llaqa: Large language and graph assistant. *arXiv preprint arXiv:2402.08170* (2024).
- [16] Zhikai Chen, Haitao Mao, Hang Li, Wei Jin, Hongzhi Wen, Xiaochi Wei, Shuaiqiang Wang, Dawei Yin, Wenqi Fan, Hui Liu, et al. 2024. Exploring the potential of large language models (llms) in learning on graphs. *ACM SIGKDD Explorations Newsletter* 25, 2 (2024), 42–61.
- [17] Zhikai Chen, Haitao Mao, Jingzhe Liu, Yu Song, Bingheng Li, Wei Jin, Bahare Fatemi, Anton Tsitsulin, Bryan Perozzi, Hui Liu, et al. 2024. Text-space Graph Foundation Models: Comprehensive Benchmarks and New Insights. *arXiv preprint arXiv:2406.10727* (2024).
- [18] Wei-Lin Chiang, Xuanqing Liu, Si Si, Yang Li, Samy Bengio, and Cho-Jui Hsieh. 2019. Cluster-gcn: An efficient algorithm for training deep and large graph convolutional networks. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 257–266.
- [19] Ganqu Cui, Jie Zhou, Cheng Yang, and Zhiyuan Liu. 2020. Adaptive graph encoder for attributed graph embedding. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 976–985.
- [20] Alex O Davies, Riku W Green, Nirav S Ajmeri, et al. 2023. Its All Graph To Me: Foundational Topology Models with Contrastive Learning on Multiple Domains. *arXiv preprint arXiv:2311.03976* (2023).
- [21] Alex O. Davies, Riku W. Green, Nirav S. Ajmeri, and Telmo M. Silva Filho. 2024. Towards Generalised Pre-Training of Graph Models. *arXiv:2311.03976 [cs.LG]*
- [22] Edsger W Dijkstra. 1959. A note on two problems in connexion with graphs. *Numerische mathematik* 1, 1 (1959), 269–271.
- [23] Vijay Prakash Dwivedi, Chaitanya K Joshi, Anh Tuan Luu, Thomas Laurent, Yoshua Bengio, and Xavier Bresson. 2023. Benchmarking graph neural networks. *Journal of Machine Learning Research* 24, 43 (2023), 1–48.
- [24] Vijay Prakash Dwivedi, Anh Tuan Luu, Thomas Laurent, Yoshua Bengio, and Xavier Bresson. 2021. Graph neural networks with learnable structural and positional representations. *arXiv preprint arXiv:2110.07875* (2021).
- [25] Vijay Prakash Dwivedi, Ladislav Rampásek, Mikhail Galkin, Ali Parviz, Guy Wolf, Anh Tuan Luu, and Dominique Beaini. 2022. Long Range Graph Benchmark. In *Thirty-sixth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*. <https://openreview.net/forum?id=in7XC5RcjEn>
- [26] Michalis Faloutsos, Petros Faloutsos, and Christos Faloutsos. 1999. On power-law relationships of the internet topology. In *Proceedings of the conference on Applications, technologies, architectures, and protocols for computer communication*. 251–262.
- [27] Wenqi Fan, Shijie Wang, Jiani Huang, Zhikai Chen, Yu Song, Wenzhuo Tang, Haitao Mao, Hui Liu, Xiaorui Liu, Dawei Yin, et al. 2024. Graph machine learning in the era of large language models (llms). *arXiv preprint arXiv:2404.14928* (2024).
- [28] Taoran Fang, Yunchao Zhang, Yang Yang, Chunping Wang, and Lei Chen. 2024. Universal prompt tuning for graph neural networks. *Advances in Neural Information Processing Systems* 36 (2024).
- [29] Bahare Fatemi, Jonathan Halcrow, and Bryan Perozzi. 2023. Talk like a graph: Encoding graphs for large language models. *arXiv preprint arXiv:2310.04560* (2023).
- [30] Tianyu Fu, Chiyue Wei, Yu Wang, and Rex Ying. 2024. Desco: Towards generalizable and scalable deep subgraph counting. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*. 218–227.
- [31] Mikhail Galkin, Xinyu Yuan, Hesham Mostafa, Jian Tang, and Zhaocheng Zhu. 2023. Towards foundation models for knowledge graph reasoning. *arXiv preprint arXiv:2310.04562* (2023).
- [32] Pierre-Louis Giscard, Nils Kriege, and Richard C Wilson. 2019. A general purpose algorithm for counting simple cycles and simple paths of any length. *Algorithmica* 81 (2019), 2716–2737.
- [33] Jiayan Guo, Lun Du, and Hengyu Liu. 2023. Gpt4graph: Can large language models understand graph structured data? an empirical evaluation and benchmarking. *arXiv preprint arXiv:2305.15066* (2023).
- [34] Will Hamilton, Zhitaoying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. *Advances in neural information processing systems* 30 (2017).
- [35] Xiaotian Han, Zhimeng Jiang, Ninghao Liu, and Xia Hu. 2022. G-mixup: Graph data augmentation for graph classification. In *International Conference on Machine Learning*. PMLR, 8230–8248.
- [36] Kaveh Hassani and Amir Hosein Khasahmadi. 2020. Contrastive multi-view representation learning on graphs. In *International conference on machine learning*. PMLR, 4116–4126.
- [37] Ruining He and Julian McAuley. 2016. Ups and downs: Modeling the visual evolution of fashion trends with one-class collaborative filtering. In *proceedings of the 25th international conference on world wide web*. 507–517.
- [38] Zhenyu Hou, Xiao Liu, Yukuo Cen, Yuxiao Dong, Hongxia Yang, Chunjie Wang, and Jie Tang. 2022. Graphmae: Self-supervised masked graph autoencoders. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 594–604.
- [39] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685* (2021).
- [40] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2020. Open graph benchmark: Datasets for machine learning on graphs. *Advances in neural information processing systems* 33 (2020), 22118–22133.
- [41] Weihua Hu, Bowen Liu, Joseph Gomes, Marinka Zitnik, Percy Liang, Vijay Pande, and Jure Leskovec. 2019. Strategies for pre-training graph neural networks. *arXiv preprint arXiv:1905.12265* (2019).
- [42] Ziniu Hu, Yuxiao Dong, Kuansan Wang, Kai-Wei Chang, and Yizhou Sun. 2020. Gpt-gnn: Generative pre-training of graph neural networks. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 1857–1867.
- [43] Qian Huang, Hongyu Ren, Peng Chen, Gregor Kržmanc, Daniel Zeng, Percy S Liang, and Jure Leskovec. 2024. Prodigy: Enabling in-context learning over graphs. *Advances in Neural Information Processing Systems* 36 (2024).
- [44] George Karypis and Vipin Kumar. 1997. METIS: A software package for partitioning unstructured graphs, partitioning meshes, and computing fill-reducing orderings of sparse matrices. (1997).
- [45] Alex Kendall, Yarin Gal, and Roberto Cipolla. 2018. Multi-task learning using uncertainty to weigh losses for scene geometry and semantics. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 7482–7491.
- [46] Thomas N Kipf and Max Welling. 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907* (2016).
- [47] Thomas N Kipf and Max Welling. 2016. Variational graph auto-encoders. *arXiv preprint arXiv:1611.07308* (2016).
- [48] Boris Knyazev, Graham W Taylor, and Mohamed Amer. 2019. Understanding attention and generalization in graph neural networks. *Advances in neural information processing systems* 32 (2019).
- [49] Lecheng Kong, Jiarui Feng, Hao Liu, Chengsong Huang, Jiaxin Huang, Yixin Chen, and Muhao Zhang. 2024. Gofa: A Generative One-For-All Model for

- Joint Graph Language Modeling. *arXiv preprint arXiv:2407.09709* (2024).
- [50] Devin Kreuzer, Dominique Beaini, Will Hamilton, Vincent Létourneau, and Prudencio Tossou. 2021. Rethinking graph transformers with spectral attention. *Advances in Neural Information Processing Systems* 34 (2021), 21618–21629.
 - [51] Namkyeong Lee, Junseok Lee, and Chanyoung Park. 2022. Augmentation-free self-supervised learning on graphs. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 36. 7372–7380.
 - [52] Jure Leskovec, Jon Kleinberg, and Christos Faloutsos. 2007. Graph evolution: Densification and shrinking diameters. In *ACM Transactions on Knowledge Discovery from Data (TKDD)*, Vol. 1. ACM New York, NY, USA, 2–es.
 - [53] Jia Li, Xiangguo Sun, Yuhao Li, Zhixun Li, Hong Cheng, and Jeffrey Xu Yu. 2024. Graph Intelligence with Large Language Models and Prompt Learning. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 6545–6554.
 - [54] Pengyong Li, Jun Wang, Ziliang Li, Yixuan Qiao, Xianggen Liu, Fei Ma, Peng Gao, Seng Song, and Guotang Xie. 2021. Pairwise half-graph discrimination: A simple graph-level self-supervised strategy for pre-training graph neural networks. *arXiv preprint arXiv:2110.13567* (2021).
 - [55] Pan Li, Yanbang Wang, Hongwei Wang, and Jure Leskovec. 2020. Distance encoding: Design provably more powerful neural networks for graph representation learning. *Advances in Neural Information Processing Systems* 33 (2020), 4465–4478.
 - [56] Derek Lim, Joshua Robinson, Lingxiao Zhao, Tess Smidt, Suvrit Sra, Haggai Maron, and Stefanie Jegelka. 2022. Sign and basis invariant networks for spectral graph representation learning. *arXiv preprint arXiv:2202.13013* (2022).
 - [57] Hao Liu, Jiarui Feng, Lecheng Kong, Ningyue Liang, Dacheng Tao, Yixin Chen, and Muhao Zhang. 2023. One for all: Towards training one graph model for all classification tasks. *arXiv preprint arXiv:2310.00149* (2023).
 - [58] Renming Liu, Semih Cantürk, Olivier Lapointe-Gagné, Vincent Létourneau, Guy Wolf, Dominique Beaini, and Ladislav Rampásek. 2023. Graph Positional and Structural Encoder. *arXiv preprint arXiv:2307.07107* (2023).
 - [59] Yuanfu Lu, Xunqiang Jiang, Yuan Fang, and Chuan Shi. 2021. Learning to pre-train graph neural networks. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 35. 4276–4284.
 - [60] Liheng Ma, Chen Lin, Derek Lim, Adriana Romero-Soriano, Puneet K Dokania, Mark Coates, Philip Torr, and Ser-Nam Lim. 2023. Graph inductive biases in transformers without message passing. In *International Conference on Machine Learning*. PMLR, 23321–23337.
 - [61] Haitao Mao, Zhikai Chen, Wenzhuo Tang, Jianan Zhao, Yao Ma, Tong Zhao, Neil Shah, Michael Galkin, and Jiliang Tang. 2024. Graph Foundation Models. *arXiv preprint arXiv:2402.02216* (2024).
 - [62] Julian McAuley, Christopher Targett, Qinfeng Shi, and Anton Van Den Hengel. 2015. Image-based recommendations on styles and substitutes. In *Proceedings of the 38th international ACM SIGIR conference on research and development in information retrieval*. 43–52.
 - [63] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. 2013. Distributed representations of words and phrases and their compositionality. *Advances in neural information processing systems* 26 (2013).
 - [64] Ron Milo, Shai Shen-Orr, Shalev Itzkovitz, Nadav Kashtan, Dmitri Chklovskii, and Uri Alon. 2002. Network motifs: simple building blocks of complex networks. *Science* 298, 5594 (2002), 824–827.
 - [65] Mikolaj Mizera, Arkadii Lin, Eugene Babin, Yury Kashkur, Tatiana Sitnik, Ien An Chan, Arsen Yedige, Maksim Vendin, Shamkhal Baybekov, and Vladimir Aladinskiy. 2024. Graph Transformer Foundation Model for modeling ADMET properties. (2024).
 - [66] Noujan Pashanasangi and C Seshadhri. 2020. Efficiently counting vertex orbits of all 5-vertex subgraphs, by evoke. In *Proceedings of the 13th International Conference on Web Search and Data Mining*. 447–455.
 - [67] Nataša Pržulj. 2007. Biological network comparison using graphlet degree distribution. *Bioinformatics* 23, 2 (2007), e177–e183.
 - [68] Nataša Pržulj, Derek G Corneil, and Igor Jurisica. 2004. Modeling interactome: scale-free or geometric? *Bioinformatics* 20, 18 (2004), 3508–3515.
 - [69] Chen Qian, Huayi Tang, Zhirui Yang, Hong Liang, and Yong Liu. 2023. Can large language models empower molecular property prediction? *arXiv preprint arXiv:2307.07443* (2023).
 - [70] Jiezhong Qiu, Qibin Chen, Yuxiao Dong, Jing Zhang, Hongxia Yang, Ming Ding, Kuansan Wang, and Jie Tang. 2020. Gcc: Graph contrastive coding for graph neural network pre-training. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 1150–1160.
 - [71] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. 2021. Learning transferable visual models from natural language supervision. In *International conference on machine learning*. PMLR, 8748–8763.
 - [72] Aditya Ramesh, Mikhail Pavlov, Gabriel Goh, Scott Gray, Chelsea Voss, Alec Radford, Mark Chen, and Ilya Sutskever. 2021. Zero-shot text-to-image generation. In *International conference on machine learning*. Pmlr, 8821–8831.
 - [73] Ladislav Rampásek, Michael Galkin, Vijay Prakash Dwivedi, Anh Tuan Luu, Guy Wolf, and Dominique Beaini. 2022. Recipe for a general, powerful, scalable graph transformer. *Advances in Neural Information Processing Systems* 35 (2022), 14501–14515.
 - [74] Fan-Yun Sun, Jordan Hoffmann, Vikas Verma, and Jian Tang. 2019. Infograph: Unsupervised and semi-supervised graph-level representation learning via mutual information maximization. *arXiv preprint arXiv:1908.01000* (2019).
 - [75] Mengying Sun, Jing Xing, Huijun Wang, Bin Chen, and Jiayu Zhou. 2021. MoCL: data-driven molecular fingerprint via knowledge-aware contrastive learning from molecular graph. In *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining*. 3585–3594.
 - [76] Damian Szklarczyk, Annika L Gable, David Lyon, Alexander Junge, Stefan Wyder, Jaime Huerta-Cepas, Milan Simonovic, Nadezhda T Doncheva, John H Morris, Peer Bork, et al. 2019. STRING v11: protein–protein association networks with increased coverage, supporting functional discovery in genome-wide experimental datasets. *Nucleic acids research* 47, D1 (2019), D607–D613.
 - [77] Lubos Takac and Michal Zabovsky. 2012. Data analysis in public social networks. In *International scientific conference and international workshop present day trends of innovations*, Vol. 1.
 - [78] Jiabin Tang, Yuhao Yang, Wei Wei, Lei Shi, Lixin Su, Suqi Cheng, Dawei Yin, and Chao Huang. 2023. Graphgpt: Graph instruction tuning for large language models. *arXiv preprint arXiv:2310.13023* (2023).
 - [79] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shriti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).
 - [80] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
 - [81] Petar Velickovic, William Fedus, William L Hamilton, Pietro Liò, Yoshua Bengio, and R Devon Hjelm. 2019. Deep graph infomax. *ICLR (Poster)* 2, 3 (2019), 4.
 - [82] Bugeun Wang, Yichao Ji, Meng Jiang, Yu Qian, Zeyu Yang, Boya Li, Gerard de Melo, Yujiu Li, and Bryan Hooi. 2024. NLGraph: A Benchmark for Natural Language Reasoning on Graphs. *arXiv preprint arXiv:2402.09842* (2024).
 - [83] Chun Wang, Shirui Pan, Guodong Long, Xingquan Zhu, and Jing Jiang. 2017. Mgae: Marginalized graph autoencoder for graph clustering. In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management*. 889–898.
 - [84] Heng Wang, Shangbin Feng, Tianxing He, Zhaoxuan Tan, Xiaochuan Han, and Yulia Tsvetkov. 2024. Can language models solve graph problems in natural language? *Advances in Neural Information Processing Systems* 36 (2024).
 - [85] Haorui Wang, Haoteng Yin, Muhao Zhang, and Pan Li. 2022. Equivariant and stable positional encoding for more powerful graph neural networks. *arXiv preprint arXiv:2203.00199* (2022).
 - [86] Zhenqin Wu, Bharath Ramsundar, Evan N Feinberg, Joseph Gomes, Caleb Geniesse, Aneesh S Pappu, Karl Leswing, and Vijay Pande. 2018. MoleculeNet: a benchmark for molecular machine learning. *Chemical science* 9, 2 (2018), 513–530.
 - [87] Jun Xia, Chengshuai Zhao, Bozhen Hu, Zhangyang Gao, Cheng Tan, Yue Liu, Siyuan Li, and Stan Z Li. 2022. Mole-bert: Rethinking pre-training graph neural networks for molecules. In *The Eleventh International Conference on Learning Representations*.
 - [88] Lianghao Xia and Chao Huang. 2024. AnyGraph: Graph Foundation Model in the Wild. *arXiv preprint arXiv:2408.10700* (2024).
 - [89] Lianghao Xia, Ben Kao, and Chao Huang. 2024. Opengraph: Towards open graph foundation models. *arXiv preprint arXiv:2403.01121* (2024).
 - [90] Dongkuan Xu, Wei Cheng, Dongsheng Luo, Haifeng Chen, and Xiang Zhang. 2021. Infogcl: Information-aware graph contrastive learning. *Advances in Neural Information Processing Systems* 34 (2021), 30414–30425.
 - [91] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2018. How powerful are graph neural networks? *arXiv preprint arXiv:1810.00826* (2018).
 - [92] Minghao Xu, Hang Wang, Bingbing Ni, Hongyu Guo, and Jian Tang. 2021. Self-supervised graph-level representation learning with local and global structure. In *International Conference on Machine Learning*. PMLR, 11548–11558.
 - [93] Dongchao Yang, Jinchuan Tian, Xu Tan, Rongjie Huang, Songxiang Liu, Xuankai Chang, Jiatong Shi, Sheng Zhao, Jiang Bian, Xixin Wu, et al. 2023. Uniaudio: An audio foundation model toward universal audio generation. *arXiv preprint arXiv:2310.00704* (2023).
 - [94] Zhilin Yang, William Cohen, and Ruslan Salakhudinov. 2016. Revisiting semi-supervised learning with graph embeddings. In *International conference on machine learning*. PMLR, 40–48.
 - [95] Ruosong Ye, Caiqi Zhang, Runhui Wang, Shuyuan Xu, and Yongfeng Zhang. 2023. Natural language is all a graph needs. *arXiv preprint arXiv:2308.07134* (2023).
 - [96] Chengxuan Ying, Tianle Cai, Shengjie Luo, Shuxin Zheng, Guolin Ke, Di He, Yanming Shen, and Tie-Yan Liu. 2021. Do transformers really perform badly for graph representation? *Advances in neural information processing systems* 34 (2021), 28877–28888.
 - [97] Zhitao Ying, Andrew Wang, Jiaxuan You, Chengtao Wen, Arquimedes Canedo, and Jure Leskovec. 2020. Neural subgraph matching. (2020).

- [98] Yuning You, Tianlong Chen, Yongduo Sui, Ting Chen, Zhangyang Wang, and Yang Shen. 2020. Graph contrastive learning with augmentations. *Advances in neural information processing systems* 33 (2020), 5812–5823.
- [99] Jiaoheng Zhang, Ali Maatouk, Jialin Chen, Ngoc Bui, Qianqian Xie, Leandros Tassioulas, Hua Xu, Jie Shao, and Rex Ying. 2025. Litfin: A retrieval augmented structure-aware foundation model for citation graphs. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 3728–3739.
- [100] Mengmei Zhang, Mingwei Sun, Peng Wang, Shen Fan, Yanhu Mo, Xiaoxiao Xu, Hong Liu, Cheng Yang, and Chuan Shi. 2024. GraphTranslator: Aligning Graph Model to Large Language Model for Open-ended Tasks. In *Proceedings of the ACM on Web Conference 2024*. 1003–1014.
- [101] Haihong Zhao, Aochuan Chen, Xiangguo Sun, Hong Cheng, and Jia Li. 2024. All in one and one for all: A simple yet effective method towards cross-domain graph pretraining. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4443–4454.
- [102] Jianan Zhao, Hesham Mostafa, Michael Galkin, Michael Bronstein, Zhaocheng Zhu, and Jian Tang. 2024. GraphAny: A Foundation Model for Node Classification on Any Graph. *arXiv preprint arXiv:2405.20445* (2024).
- [103] Jianan Zhao, Le Zhuo, Yikang Shen, Meng Qu, Kai Liu, Michael Bronstein, Zhaocheng Zhu, and Jian Tang. 2023. GraphText: Graph reasoning in text space. *arXiv preprint arXiv:2310.01089* (2023).
- [104] Qi Zhu, Da Zheng, Xiang Song, Shichang Zhang, Bowen Jin, Yizhou Sun, and George Karypis. 2024. Parameter-Efficient Tuning Large Language Models for Graph Representation Learning. *arXiv preprint arXiv:2404.18271* (2024).
- [105] Wenhao Zhu, Tianyu Wen, Guojie Song, Liang Wang, and Bo Zheng. 2023. On structural expressive power of graph transformers. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 3628–3637.

A Methodologies

A.1 Dataset

Table 8: Dataset Information. *class.* represents classification task and *reg.* represents regression task.

Dataset	Num. graphs	Num. nodes	Num. edges	Pred. level	Pred. task	Num. tasks	Metric
MolPCBA	437,929	25.97	28.11	graph	class. (binary)	128	AP
MolHIV	41,127	25.51	27.46	graph	class. (binary)	1	AUROC
MNIST	70,000	70.57	281.65	graph	class. (10-way)	1	ACC
Peptides-func	15,535	150.94	153.65	graph	class. (binary)	10	AP
Peptides-struct	15,535	150.94	153.65	graph	reg.	11	MAE
ogbn-proteins	1	132,534	39,561,252	node	class. (binary)	112	AUROC
Pokec	1	1,632,803	30,622,564	node	class. (binary)	1	ACC
ogbn-arxiv	1	169,343	1,166,243	node	class. (40-way)	1	ACC
ogbn-products	1	2,449,029	61,859,140	node	class. (47-way)	1	ACC
ZINC	249,456	23.2	49.8	graph	reg.	1	MAE
PubMed	19,717	88,648	500	node	class. (3-way)	1	ACC
CIFAR10	60,000	117.6	941.2	graph	class. (10-way)	1	ACC

Table 9: Dataset statistics of three categories from Amazon e-commerce networks

	# nodes	# edges	avg. degree	avg. # tokens
Clothing	469,274	2,578,746	10.99	117.83
Home	453,121	3,732,948	16.48	133.94
Sports	293,712	2,390,076	16.27	125.08

A.2 Pre-training Metric

We use an accuracy metric to measure Community Detection and Graph Contrastive Learning and use mean squared error (MSE) to measure the performance of Shortest Path Distance regression and Motif Counting.

- For the community detection task, we set ϵ as 1. The predicted label between the i -th node and the j -th node $\hat{Y}_{i,j}^c$ is 1 if $\text{sim}(i, j) \geq 0.5$ and 0 otherwise. Accuracy is calculated by comparing the

predicted label $\hat{Y}_{i,j}^c$ with the ground truth labels $Y_{i,j}^c$ and is defined as the proportion of correctly predicted labels out of all possible node pairs:

$$\text{Accuracy(CD)} = \frac{\sum_{(i,j) \in V} \mathbb{1}(\hat{Y}_{i,j}^c = Y_{i,j}^c)}{|V|(|V| - 1)/2}, \quad (3)$$

where $\mathbb{1}(\cdot)$ is an indicator function and $|V|(|V| - 1)/2$ is the total number of unique node pairs in the graph. This metric effectively measures how well the model can identify community structures by correctly classifying node pairs as being in the same community or in different communities.

- For the graph contrastive learning task, we evaluate pre-training performance using the accuracy metric, which measures the model’s ability to correctly classify graphs as originating from the same or different datasets. The accuracy is computed by:

$$\text{Accuracy(GCL)} = \frac{\sum_{i,j} \mathbb{1}(\hat{Y}_{G_i, G_j} = Y_{G_i, G_j})}{N}, \quad (4)$$

where $\hat{Y}_{G_i, G_j}$ is the predicted label indicating whether graph G_i and G_j are from the same dataset and Y_{G_i, G_j} is the ground truth label. $\hat{Y}_{G_i, G_j}$ is 1 if $\text{sim}(\mathbf{z}_{G_i}, \mathbf{z}_{G_j}) \geq 0$ and 0 otherwise. N is the total number of evaluated graph pairs.

- For shortest path distance regression, the mean squared error (MSE) is used as a metric, which is defined as:

$$\text{MSE(SPD)} = \frac{1}{|E|} \sum_{(i,j) \in E} (h_{\text{SPD}}(\mathbf{P}_i^L \| \mathbf{P}_j^L) - \text{SPD}_{i,j})^2. \quad (5)$$

The ground truth SPD is normalized by the graph diameter to ensure scale consistency and training stability.

- For the motif counting task, the mean absolute error is used as a metric, which is defined as:

$$\text{MAE(MC)} = \frac{1}{|V|} \sum_{i \in V} \|h_{\text{MC}}(\mathbf{P}_i^L) - Y_i^m\|_1, \quad (6)$$

where Y_i^m is the pre-computed label for the i -th node.

A.3 Uncertainty-based Loss Weighing

The scale of the loss of different tasks can be different, causing the overall loss to be dominated by a certain task, and ultimately the loss of the other tasks cannot affect the learning process of the network-sharing layers. We use the uncertainty-based loss-weighting method [45] to automatically balance the four pre-training tasks and unify the different scales. Moreover, the uncertainty value reflects the contribution of each task towards the overall pre-training process. A higher uncertainty value indicates a lower contribution [45]. Let σ_τ and $\mathcal{L}_\tau$ represent the task-specific uncertainty value for the task τ . The overall pre-training loss is computed by:

$$\mathcal{L} = \frac{1}{\sigma_{\text{SPD}}^2} \mathcal{L}_{\text{SPD}} + \frac{1}{\sigma_{\text{MC}}^2} \mathcal{L}_{\text{MC}} + \frac{1}{\sigma_{\text{CD}}^2} \mathcal{L}_{\text{CD}} + \frac{1}{\sigma_{\text{GCL}}^2} \mathcal{L}_{\text{GCL}} + \log \sigma_{\text{SPD}} + \log \sigma_{\text{MC}} + \log \sigma_{\text{CD}} + \log \sigma_{\text{GCL}}. \quad (7)$$

A.4 Pre-training Setting

The pre-training stage is conducted on the standard train/validation/test splits of the pre-training datasets. The dimension of initial encoding d is set as 8. We try GatedGCN [9], GCN [46] and GIN [91]

as the message-passing layers in the GPS. The number of GPS layers is tuned in the range of [4, 16] and the number of heads is tuned within {4, 8, 16}. The hidden dimension is tuned {32, 64, 128, 256}. The output PSE dimension is in {32, 64}. The temperature τ is set as 0.1 and the margin is 0. We use Adam as the optimizer with an initial learning rate of 0.001, and the batch size is set as 256. The maximum training epochs is 100. An early stopping strategy is used to mitigate overfitting. The experiments are implemented on the NVIDIA A40 48GB GPU.

B More Experimental Results

B.1 Integration with Pre-trained Models

All models are fine-tuned, trained or tested using five different seeds from 42 to 46, with the results averaged. Additionally, for the results of our generated PSE, we select three different seeds to obtain three GFSE checkpoints. Each GFSE is used to run the downstream task five times with the aforementioned seeds (42–46), and all results are averaged.

For the training from scratch setting, we adopt and modify the code base from GPS [73]¹. RWSE and LapPE are of dimension 32 for molecule benchmark in Table 4 across all the datasets. Given a graph $G(V, E)$, we directly concatenate the PSE and the raw node feature as the new input feature, then send them into the very beginning of a model with randomly initialized parameters, which is as follows:

$$X' = \text{concat}(X, \text{PSE}); \quad \hat{y} = \text{MLP}[\text{pooling}, [\text{GraphModel}(X')]]$$

where X denotes the raw node feature of the input graph, X' is the input feature augmented with structural information, GraphModel denotes our backbones GNN or GPS, and the read-out layer consists of pooling and MLPs to obtain the final prediction.

For the fine-tuning setting in the molecule benchmark, we concatenate the node encodings obtained from a pre-trained model with the extra structural features. Then we send the concatenated features into the final read-out layers for the final prediction. Note that during the fine-tuning process, the parameters of the entire model (both the pre-trained model and the read-out layer) are continuously updated.

$$X' = \text{GNN}(X); \quad \hat{y} = \text{MLP}[\text{pooling}[\text{concat}(X', \text{PSE})]], \quad (8)$$

where X' denotes the latent node features output from the pre-trained models, PSE denotes the extra structural feature generated by GFSE, the read-out layer consists of pooling and MLPs, where the hyperparameters follow exactly as [87].

B.2 Downstream Evaluation Performance

From Table 10, we observe that the optimal selection of structural encoding typically varies across different datasets and base models. For example, RWSE tends to be more beneficial than LapPE for small molecular graph learning (e.g., MolPCBA and ZINC), whereas the opposite is generally true on Peptides. Notably, the performance gains are most pronounced when integrating PSE with Transformer, demonstrating its critical role in compensating for the absence of inherent structural sensitivity in Transformers. The last row shows the average improvement (%) brought by GFSE on base models.

¹<https://github.com/rampasek/GraphGPS>

Table 10: Performance on MolPCBA, ZINC (subset), Peptides-func and Peptides-struct. The best results in each dataset are bolded.

	MolPCBA AP $\uparrow$	ZINC MAE $\downarrow$	Peptides-func AP $\uparrow$	Peptides-struct MAE $\downarrow$
GCN	0.2424 $\pm$ 0.0034	0.3670 $\pm$ 0.0110	0.5930 $\pm$ 0.0023	0.3496 $\pm$ 0.0013
GCN+LapPE	0.2417 $\pm$ 0.0047	0.2052 $\pm$ 0.0132	0.6021 $\pm$ 0.0051	0.2688 $\pm$ 0.0027
GCN+RWSE	0.2438 $\pm$ 0.0028	0.1741 $\pm$ 0.0528	0.5827 $\pm$ 0.0046	0.3270 $\pm$ 0.0019
GCN+GFSE	0.1958 $\pm$ 0.0074	0.1218 $\pm$ 0.0613	0.5959 $\pm$ 0.0034	0.2710 $\pm$ 0.0041
GCN+GFSE	0.2477 $\pm$ 0.0021	0.1237 $\pm$ 0.0428	0.6131 $\pm$ 0.0074	0.2513 $\pm$ 0.0054
GIN	0.2703 $\pm$ 0.0023	0.5260 $\pm$ 0.0510	0.5498 $\pm$ 0.0079	0.3547 $\pm$ 0.0045
GIN+LapPE	0.2701 $\pm$ 0.0013	0.2203 $\pm$ 0.0386	0.5323 $\pm$ 0.0083	0.2650 $\pm$ 0.0041
GIN+RWSE	0.2781 $\pm$ 0.0031	0.1731 $\pm$ 0.0614	0.5410 $\pm$ 0.0068	0.3282 $\pm$ 0.0037
GIN+GFSE	0.2765 $\pm$ 0.0073	0.2162 $\pm$ 0.0429	0.5389 $\pm$ 0.0094	0.2581 $\pm$ 0.0046
GIN+GFSE	0.2839 $\pm$ 0.0046	0.1689 $\pm$ 0.0524	0.5532 $\pm$ 0.0103	0.2674 $\pm$ 0.0039
Transformer (TF)	0.0808 $\pm$ 0.0117	0.6943 $\pm$ 0.0328	0.4800 $\pm$ 0.0076	0.4192 $\pm$ 0.0028
TF+LapPE	0.1784 $\pm$ 0.0329	0.5101 $\pm$ 0.0724	0.6307 $\pm$ 0.0091	0.2514 $\pm$ 0.0031
TF+RWSE	0.2083 $\pm$ 0.0674	0.2193 $\pm$ 0.0640	0.6326 $\pm$ 0.0028	0.3342 $\pm$ 0.0028
TF+GFSE	0.2040 $\pm$ 0.0531	0.1883 $\pm$ 0.0263	0.6534 $\pm$ 0.0041	0.2479 $\pm$ 0.0068
TF+GFSE	0.2376 $\pm$ 0.0342	0.1548 $\pm$ 0.0796	0.6642 $\pm$ 0.0025	0.2436 $\pm$ 0.0071
GPS	0.2869 $\pm$ 0.0045	0.1182 $\pm$ 0.0049	0.6535 $\pm$ 0.0041	0.2500 $\pm$ 0.0012
GPS+LapPE	0.2939 $\pm$ 0.0016	0.1078 $\pm$ 0.0084	0.6494 $\pm$ 0.0037	0.2501 $\pm$ 0.0026
GPS+RWSE	0.2907 $\pm$ 0.0028	0.0700 $\pm$ 0.0040	0.6603 $\pm$ 0.0085	0.2739 $\pm$ 0.0063
GPS+GFSE	0.2911 $\pm$ 0.0036	0.0648 $\pm$ 0.0030	0.6688 $\pm$ 0.0151	0.2464 $\pm$ 0.0025
GPS+GFSE	0.2916 $\pm$ 0.0061	0.0613 $\pm$ 0.0026	0.6874 $\pm$ 0.0120	0.2474 $\pm$ 0.0051
GFSE Imp.(%)	32.60	76.43	2.78	42.47

The consistent improvements across different settings underscore the robustness and generalizability of GFSE, especially in contexts where traditional PSE fails to deliver.

B.3 Efficiency Evaluation

During the pre-training on eight datasets, the average time is around 30 to 40 minutes for each epoch with a single NVIDIA A40 48GB GPU. Total training time is less than two days, which is relatively efficient for a comprehensive multi-dataset pre-training process.

Table 11: Runntimes (s) of PSE computation on random synthetic graph with increasing numbers of nodes

PSE / Graph size	100	300	500	1000
LapPE	2	9.25	34	155
RWSE	2	9.76	31.48	207
Pre-computation	0.0007	0.001	0.003	0.006
GFSE Inference	0.908	3.958	10.770	48.106

We compare the inference efficiency of GFSE with handcrafted positional encodings, such as LapPE and RWSE in Table 11 and Table 12. Specifically, we generate 1,000 synthetic Erdos-Rényi graphs for various graph sizes (100, 300, 500, and 1,000 nodes) and evaluate the time required for pre-computation and inference.

Table 12: Runntimes (s) on real-world graph dataset

Dataset	ZINC	MolHIV	MolPCBA	Peptides	MNIST	CIFAR10
LapPE	25 sec	37 sec	6.13 min	73 sec	96 sec	2.55 min
RWSE	11 sec	58 sec	8.33 min	-	-	-
GFSE	4.17 sec	17.23 sec	2.97 min	15.21 sec	49.38 sec	1.27 min

As shown in the Table 11, both LapPE and RWSE exhibit significant increases in computation time as the graph size grows. Pre-computation times required by GFSE inference remain minimal for all graph sizes, underlining the model’s efficiency in this phase. In Table 12, we observe that GFSE demonstrates superior scalability in inference, making it a more efficient option for large-scale graph processing.